Ecole Supérieure d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

Ing3 – Web Dev I

TP5 : PDO – Gestion d'un catalogue de films

Objectif du TP

L'objectif de cet atelier est de maîtriser l'utilisation de la librairie **PDO** en PHP. Vous développerez une mini-application de gestion d'un catalogue de films en implémentant les opérations **CRUD** (Create, Read, Update, Delete) à l'aide de requêtes préparées et d'une architecture orientée objet.

Étape 1 : Création de la base de données et préparation du projet

— Créez la base de données `CinemaBD` via **phpMyAdmin**.

— Créez la table `films` selon la structure suivante :

#	Nom	Type	Interclassement	Null	Défaut	Extra
1	`id`	`INT(11)`	–	Non	Aucune	`AUTO_INCREMENT`
2	`titre`	`VARCHAR(100)`	utf8mb4_general_ci	Non	Aucune	
3	`realisateur`	`VARCHAR(60)`	utf8mb4_general_ci	Non	Aucune	
4	`annee`	`YEAR`	–	Non	Aucune	
5	`genre`	`VARCHAR(30)`	utf8mb4_general_ci	Non	Aucune	
6	`note`	`DECIMAL(3,1)`	–	Non	Aucune	

Remarque : Le champ `note` est de type `DECIMAL(3,1)`, ce qui permet des valeurs comme `7.5` ou `9.0`. Le champ `annee` est de type `YEAR` (pas `INT`) – tenez-en compte lors du binding PDO.

— Créez un nouveau projet PHP `TP5` dans votre dossier `htdocs`.

— Le fichier `Film.php` vous est fourni ci-dessous sous forme de **squelette**. Copiez-le dans votre projet – **ne modifiez pas les signatures des méthodes**.

Listing 1 – Film.php – squelette fourni

```php
<?php
class Film {

    // -- Attributs ----------------------------------------------------
    private int    $id;
    private string $titre;
    private string $realisateur;
    private int    $annee;       // YEAR stocke en entier
```

```
    private string $genre;
    private float  $note;       // DECIMAL(3,1)

    // -- Constructeur ----------------------------------------------------
    public function __construct(
        string $titre,
        string $realisateur,
        int    $annee,
        string $genre,
        float  $note,
        int    $id = 0
    ) {
        $this->titre       = $titre;
        $this->realisateur = $realisateur;
        $this->annee       = $annee;
        $this->genre       = $genre;
        $this->note        = $note;
        $this->id          = $id;
    }

    // -- Accesseurs ------------------------------------------------------
    public function getId()          : int    { return $this->id; }
    public function getTitre()       : string { return $this->titre; }
    public function getRealisateur() : string { return $this->realisateur; }
    public function getAnnee()       : int    { return $this->annee; }
    public function getGenre()       : string { return $this->genre; }
    public function getNote()        : float  { return $this->note; }

    // -- Modificateurs (a completer) -------------------------------------
    public function setTitre(string $titre)             : void { /* TODO */ }
    public function setRealisateur(string $realisateur) : void { /* TODO */ }
    public function setAnnee(int $annee)                : void { /* TODO */ }
    public function setGenre(string $genre)             : void { /* TODO */ }
    public function setNote(float $note)                : void { /* TODO */ }

    // -- Methodes PDO a implementer --------------------------------------

    public function save(PDO $pdo): bool {
        // TODO : inserer $this dans la table films
    }

    public function getAll(PDO $pdo): array {
        // TODO : retourner un tableau d'objets Film tries par note desc
    }

    public function findById(PDO $pdo, int $id): ?Film {
```

```
        // TODO : retourner l'objet Film ou null si inexistant
    }

    public function update(PDO $pdo): bool {
        // TODO : mettre a jour tous les champs de $this (sauf id)
    }

    public function delete(PDO $pdo, int $id): bool {
        // TODO : supprimer le film dont l'id est $id
    }

    //--Devoir--------------------------------
    public static function getByGenre(PDO $pdo, string $genre): array {
        // TODO : retourner les films du genre donne (requete preparee)
        //        tries par note decroissante
    }

    public function getStats(PDO $pdo): array {
        // TODO : retourner ['total', 'note_moyenne', 'meilleur_film']
        //        en une seule requete SQL
    }
}
```

Étape 2 : Connexion PDO centralisée

— Créez un fichier `connexion.php` qui retourne un objet `PDO` configuré pour la base `CinemaBD`.

— **Tous les fichiers du projet doivent utiliser ce fichier** via `require_once`.

Étape 3 : Ajout d'un film (save)

— Implémentez la méthode `save(PDO $pdo): bool` dans la classe `Film`. Elle doit utiliser une **requête préparée** pour insérer un nouveau film dans la BD.

— Utilisez `bindValue()` pour lier les paramètres. Précisez le type PDO correct pour chaque champ (`PDO::PARAM_STR`, `PDO::PARAM_INT`).

— Créez le fichier `ajout.php` qui :

  — Affiche un formulaire HTML de saisie des informations d'un film,

  — Traite la soumission du formulaire en instanciant un objet `Film` puis en appelant `save($pdo)`,

  — Affiche un message de succès ou d'erreur selon le retour de `save()`.

— Vérifiez l'insertion dans **phpMyAdmin**.

Contrainte imposée : La méthode `save()` de la classe `Film` doit respecter **exactement** la signature suivante – ne la modifiez pas :

```
public function save(PDO $pdo): bool
```

Elle reçoit l'objet de connexion en paramètre et retourne `true` si l'insertion réussit, `false` sinon. De même, les méthodes de lecture et de mise à jour respecteront des signatures imposées décrites aux étapes suivantes.

Étape 4 : Affichage du catalogue (`getAll`)

— Implémentez dans la classe `Film` la méthode statique suivante :

```
public function getAll(PDO $pdo): array
```

— Cette méthode retourne un tableau d'**objets** `Film`. Utilisez `PDO::FETCH_ASSOC` pour le peuplement automatique des objets.

— Créez le fichier `catalogue.php` qui :

  — Appelle `Film::getAll($pdo)`,

  — Affiche les films dans un tableau HTML avec les colonnes : *Titre, Réalisateur, Année, Genre, Note*,

  — Ajoute sur chaque ligne deux liens : **Modifier** (redirige vers `modifierFilm.php?id=X`) et **Supprimer** (redirige vers `supprimerFilm.php?id=X`).

Attention : Le tri des films dans le catalogue doit se faire par `note` décroissante (les mieux notés en premier). Gérez ce tri **directement dans la requête SQL**, pas en PHP.

Étape 5 : Suppression d'un film (`delete`)

— Implémentez la méthode suivante dans la classe `Film` :

```
public function delete(PDO $pdo, int $id): bool
```

— Créez le fichier `supprimerFilm.php` qui récupère l'`id` passé en GET, appelle `Film::delete()`, affiche un message de résultat, et permet de redirige vers `catalogue.php`.

Étape 6 : Modification d'un film (`findById` et `update`)

— Implémentez les deux méthodes suivantes dans la classe `Film` :

```
public function findById(PDO $pdo, int $id): ?Film

public function update(PDO $pdo): bool
```

— `findById` retourne un objet `Film` si l'enregistrement existe, ou `null` sinon.

— `update` met à jour **tous les champs** du film en base (sauf l'`id`) à partir des attributs de l'objet courant.

— Créez le fichier `modifierFilm.php` qui :

  — avec **POST** : met à jour les données via `update()` et redirige vers le catalogue.

  — Affiche un formulaire HTML de saisie des informations d'un film (Le même utilise dans étape 3)

— Si `findById` retourne `null`, affichez un message d'erreur approprié.

Étape 7 : Recherche par genre (`getByGenre`) *(Devoir)*

— Ajoutez dans la classe `Film` la méthode statique suivante :

```
public function getByGenre(PDO $pdo, string $genre): array
```

— Elle retourne uniquement les films du genre spécifié, toujours triés par note décroissante.

— La requête doit être **préparée** (pas de concaténation directe de la variable `$genre` dans le SQL).

— Ajoutez sur la page `catalogue.php` un formulaire de filtrage par genre (liste déroulante ou champ texte) qui utilise cette méthode lorsqu'un genre est sélectionné.

Étape 8 : Statistiques – méthode `getStats` *(Devoir)*

— Implémentez la méthode suivante :

```
public function getStats(PDO $pdo): array
```

— Elle doit retourner un tableau associatif contenant :

  — `'total'` : nombre total de films,

  — `'note_moyenne'` : note moyenne arrondie à 1 décimale,

  — `'meilleur_film'` : titre du film avec la note la plus élevée.

— Ces statistiques doivent être obtenues en **une seule requête SQL** utilisant les fonctions d'agrégation (`COUNT`, `AVG`, `MAX` avec sous-requête).

— Affichez ces statistiques dans un encart en haut de la page `catalogue.php`.

Rappel important : Les signatures des méthodes sont **imposées** et ne doivent pas être modifiées. Toute implémentation qui ne respecte pas la signature exacte (nom, paramètres, type de retour) sera considérée comme incorrecte, même si le code fonctionne.